

Backend Scaffold

Backend Scaffold

Overview

This document provides a complete backend scaffold for the Agent Dashboard MVP. It includes folder structure, TypeScript starter files, interfaces, and TODO markers for implementation. This scaffold is designed to align with the /spec contracts and provide a clean foundation for backend development.

Folder Structure

```
/backend
  /api
    agents.ts
    sessions.ts
  /runtime
    agentRuntimeAdapter.ts
    sessionManager.ts
    eventStreamServer.ts
  /models
    Agent.ts
    Session.ts
    Events.ts
  /utils
    EventSerializer.ts
    WebSocketConnection.ts
  server.ts
```

File: server.ts

```
import express from "express";
import { createServer } from "http";
import { WebSocketServer } from "ws";
import agentsRouter from "../api/agents";
import sessionsRouter from "../api/sessions";
import { setupEventStreamServer } from "../runtime/eventStreamServer";

const app = express();
app.use(express.json());

app.use("/agents", agentsRouter);
app.use("/sessions", sessionsRouter);

const httpServer = createServer(app);
const wss = new WebSocketServer({ server: httpServer });
setupEventStreamServer(wss);

httpServer.listen(3001, () => {
  console.log("Backend server running on port 3001");
});
```

File: /api/agents.ts

```
import { Router } from "express";
import { AgentRegistry } from "../../models/Agent";

const router = Router();

router.get("/list", (req, res) => {
  res.json(AgentRegistry.list());
});

router.get("/:agentId/schema", (req, res) => {
  const agent = AgentRegistry.get(req.params.agentId);
  if (!agent) return res.status(404).json({ error: "Agent not found" });
  res.json(agent.schema);
});

export default router;
```

File: /api/sessions.ts

```
import { Router } from "express";
import { SessionManager } from "../runtime/sessionManager";

const router = Router();

router.post("/create", async (req, res) => {
  const { agentId, config, inputDraft } = req.body;
  const session = SessionManager.createSession(agentId, config, inputDraft);
  res.json({ sessionId: session.id, state: session.state });
});

router.post("/:sessionId/run", async (req, res) => {
  const { input, config } = req.body;
  SessionManager.runSession(req.params.sessionId, input, config);
  res.json({ status: "started" });
});

router.post("/:sessionId/stop", async (req, res) => {
  const ok = await SessionManager.stopSession(req.params.sessionId);
  res.json({ status: ok ? "stopped" : "failed" });
});

export default router;
```

File: /runtime/sessionManager.ts

```
import { Session } from "../models/Session";
import { AgentRuntimeAdapter } from "../agentRuntimeAdapter";
import { EventStreamServer } from "../eventStreamServer";

export const SessionManager = {
  sessions: new Map<string, Session>(),

  createSession(agentId: string, config: any, inputDraft: any): Session {
    const session = new Session(agentId, config, inputDraft);
    this.sessions.set(session.id, session);
    return session;
  },

  async runSession(sessionId: string, input: any, config: any) {
    const session = this.sessions.get(sessionId);
    if (!session) throw new Error("Session not found");

    session.transition("running");
    EventStreamServer.emitLifecycleEvent(sessionId, "SESSION_STARTED", { input, config });

    const runtime = AgentRuntimeAdapter.getRuntime(session.agentId);

    for await (const event of runtime.istreamEvents(input, { sessionId, config })) {
      EventStreamServer.emitEvent(event);
      session.applyEvent(event);
    }
  },

  async stopSession(sessionId: string): Promise<boolean> {
    const session = this.sessions.get(sessionId);
    if (!session) return false;

    const runtime = AgentRuntimeAdapter.getRuntime(session.agentId);
    return runtime.abort({ sessionId });
  }
};
```

File: /runtime/eventStreamServer.ts

```
import { WebSocketServer } from "ws";
import { serializeEvent } from "../utils/EventSerializer";

export const EventStreamServer = {
  wss: null as WebSocketServer | null,

  setup(wss: WebSocketServer) {
    this.wss = wss;
  },

  emitEvent(event: any) {
    if (!this.wss) return;
    const payload = serializeEvent(event);
    this.wss.clients.forEach(client => {
      client.send(payload);
    });
  },

  emitLifecycleEvent(sessionId: string, type: string, data: any) {
    this.emitEvent({ sessionId, type, ...data, timestamp: Date.now() });
  }
};

export function setupEventStreamServer(wss: WebSocketServer) {
  EventStreamServer.setup(wss);
}
```

File: /runtime/agentRuntimeAdapter.ts

```
import { AgentRegistry } from "../models/Agent";

export const AgentRuntimeAdapter = {
  getRuntime(agentId: string) {
    const agent = AgentRegistry.get(agentId);
    if (!agent) throw new Error("Agent not found");
    return agent.runtime;
  }
};
```

File: /models/Session.ts

```
import { v4 as uuid } from "uuid";

export class Session {
  id: string;
  agentId: string;
  state: string = "idle";
  config: any;
  inputDraft: any;

  constructor(agentId: string, config: any, inputDraft: any) {
    this.id = uuid();
    this.agentId = agentId;
    this.config = config;
    this.inputDraft = inputDraft;
  }

  transition(newState: string) {
    this.state = newState;
  }

  applyEvent(event: any) {
    if (event.type === "SESSION_STATE_CHANGED") {
      this.state = event.to;
    }
  }
}
```

File: /models/Agent.ts

```
export const AgentRegistry = {
  agents: new Map<string, any>(),

  register(agent: any) {
    this.agents.set(agent.id, agent);
  },

  get(agentId: string) {
    return this.agents.get(agentId);
  },

  list() {
    return Array.from(this.agents.values()).map(a => ({
      id: a.id,
      name: a.name,
      description: a.description
    }));
  }
};
```

File: /models/Events.ts

```
export type AgentEvent = {
  sessionId: string;
  type: string;
  timestamp: number;
  [key: string]: any;
};
```

File: /utils/EventSerializer.ts

```
export function serializeEvent(event: any): string {
  return JSON.stringify(event);
}
```

File: /utils/WebSocketConnection.ts

```
import WebSocket from "ws";

export function createWebSocketConnection(url: string): WebSocket {
  const ws = new WebSocket(url);

  ws.on("open", () => console.log("Connected to event stream"));
  ws.on("close", () => console.log("Disconnected from event stream"));
  ws.on("error", err => console.error("WebSocket error", err));

  return ws;
}
```

Summary

This backend scaffold provides:

- Complete folder structure
- Session manager
- Event streaming server
- Agent runtime adapter
- API routes
- Models and utilities

It is fully aligned with the /spec contracts and ready for implementation.